DOCUMENTO INTERNO · DIRECCIÓN

Sistema de blog de ingeniak.mx

Especificación técnica y de operación · Opción A: "el repo es el CMS" con un agente como editor · Octubre 2026

1. Resumen

El blog de ingeniak.mx se publica desde el propio repositorio del sitio (GitHub → Vercel). No se agrega ningún CMS externo: el equipo de SEO entrega **briefs** en un formato simple, un **agente de IA** los convierte al esquema del sitio y abre un Pull Request, Vercel genera una **vista previa** con el artículo ya diseñado, el equipo la revisa, y **solo se publica lo que una persona aprueba**. El diseño, el SEO técnico y los verificadores de calidad que ya tiene el sitio aplican automáticamente a cada artículo.

> **Por qué así:** el sitio ya tiene un sistema de control de calidad construido (esquema estricto por artículo, verificadores de cifras/SEO/contenido que tiran el deploy si algo está mal, y un campo `estado` que impide publicar texto no aprobado). Cualquier CMS externo lo esquivaría; este flujo lo aprovecha.

2. Arquitectura: qué existe y qué se construye

Ya existe en el repo

Pieza	Qué hace
`src/data/sitemap.json`	Una entrada por artículo (template T-ARTICLE) con todo el SEO: title, meta description, keyword principal y secundarias, JSON-LD, breadcrumbs y enlaces internos.
`src/content/paginas/art-<slug>.json`	El texto del artículo con esquema fijo: hero, respuesta directa, índice, cuerpo por bloques, FAQ (del banco único `src/data/faq.json`), autor, fuentes, relacionados y CTA. Campo `estado`: borrador | revisión | aprobado.
`ArticleTemplate.astro`	Pinta cualquier artículo con el diseño del sitio. Un artículo nuevo no requiere tocar diseño.
Verificadores del build	`npm run build` corre checks de cifras, SEO, pendientes, pesos y contenido. Si algo falla, no hay deploy.
Gate de publicación	Solo se publica un artículo con `estado: "aprobado"` y nombre en `aprobadoPor`. Un borrador puede estar en el repo sin aparecer en el sitio.

Se construye (una sola vez)

Pieza	Qué hace	Esfuerzo
`scripts/blog-nuevo.mjs`	Andamiaje: a partir de un brief en Markdown genera la entrada del sitemap + el JSON del artículo en `estado: "borrador"`, con validación del esquema.	1 sesión
`BLOG.md`	Guía en la raíz del repo: formato del brief, reglas editoriales, flujo y comandos. Es el contrato entre SEO y el agente.	Incluida
`materiales/briefs/`	Bandeja de entrada: cada brief es un archivo `YYYY-MM-DD-slug.md`. Lo que está ahí, el agente lo procesa.	Trivial
Agente editor	Sesión de Claude Code (manual) o agente programado en la nube (automático, p. ej. revisa la bandeja cada lunes). Convierte brief → JSON + PR.	Config.

3. El flujo completo

1. **SEO entrega el brief.**
   Un archivo Markdown (o Google Doc que se pega) con: título, keyword, intención de búsqueda, outline H2/H3, respuesta directa, FAQs, fuentes con URL y fecha, y enlaces internos sugeridos. Formato exacto en la guía del equipo.

2. **El agente lo maquila.**
   Corre `scripts/blog-nuevo.mjs`, completa el cuerpo respetando el texto del brief, arma JSON-LD, meta y enlaces, deja `estado: "borrador"` y abre un Pull Request. Nunca inventa cifras: si falta un dato lo anota en `pendientes`.

3. **Vercel genera el preview.**
   El PR produce una URL privada donde el artículo se ve ya diseñado, con tipografía, escenas y CTA reales. Los verificadores del build corren solos; si algo falla, el agente lo corrige antes de avisar.

4. **SEO revisa en el preview.**
   Con su checklist (texto fiel al brief, H2/H3, meta, enlaces, FAQ). Pide cambios comentando el PR o por el canal acordado; el agente los aplica y el preview se actualiza.

5. **Tú (o quien delegues) apruebas.**
   El agente pasa `estado` a `"aprobado"` con el nombre en `aprobadoPor` y se hace merge a `main`.

6. **Se publica solo.**
   Vercel despliega, el artículo entra al sitemap XML, a `llms.txt` y a los enlaces internos del hub /blog/. Sin pasos manuales.

4. Cómo se alimenta (intake)

- **Nivel 1 — Manual (arranque, recomendado ya):** el equipo de SEO manda el brief (archivo o texto) y tú lo pegas en una sesión de Claude Code diciendo "procesa este brief". Cero infraestructura.
- **Nivel 2 — Bandeja compartida:** carpeta `materiales/briefs/` en el repo o una carpeta de Google Drive. Un agente programado en la nube (cron semanal) toma los briefs nuevos y abre los PRs sin que nadie se lo pida.
- **Nivel 3 — n8n como ventanilla (opcional):** un formulario o base de Notion donde SEO captura el brief; n8n lo convierte a Markdown, lo sube a la bandeja y dispara al agente. Útil solo si el volumen crece (4+ artículos/mes).

> **Regla fija en cualquier nivel:** la automatización llega hasta el borrador y el PR. El paso a `"aprobado"` siempre lo da una persona con nombre y apellido. Nada se publica sin esa firma.

5. Cómo se mantiene

Operación mensual (≈ 1–2 h/mes de tu lado)

- **Aprobar:** revisar los previews que SEO ya validó y dar el visto bueno. Es el único paso que no se delega al agente.
- **Actualizaciones de artículos viejos:** mismo flujo con brief de actualización; el campo `actualizado` alimenta el `dateModified` del JSON-LD (señal de frescura para Google).
- **Medición:** Google Search Console sobre ingeniak.mx (impresiones, clics y posición por artículo). Revisión mensual con SEO para decidir qué se actualiza, qué se amplía y qué se deja.

Salud del sistema (automática o del agente)

- Los verificadores del build vigilan cifras sin fuente, metas fuera de medida y texto placeholder en cada deploy.
- El banco de FAQ (`src/data/faq.json`) es único: una respuesta se corrige una vez y cambia en todas las páginas que la citan.
- Enlaces internos: cada artículo nuevo debe enlazar a 2–3 páginas de servicio y recibir enlace desde el hub; el script lo deja anotado y el agente lo verifica.
- Imágenes: el agente genera/optimiza a WebP con los pesos que el build exige.

6. Gobernanza (reglas no negociables)

- Nada de cifras, testimonios, clientes o casos inventados. Toda cifra lleva fuente y fecha visibles; si el brief no la trae, el dato se omite y queda en `pendientes` .
- El agente nunca cambia `estado` a aprobado por sí solo; solo registra la aprobación que una persona dio.

- Un artículo = un PR. Nunca se mezclan artículos con cambios de diseño o de código en el mismo PR.
- El esquema de `src/content/paginas/_ESQUEMA.md` es la única forma válida de un artículo; si el esquema cambia, cambia primero ahí.

7. Roles

Rol	Responsable	Hace
Estrategia y briefs	Equipo SEO	Keywords, intención, outline, fuentes, revisión en preview.
Maquila y técnica	Agente (Claude)	Brief → JSON + sitemap, JSON-LD, enlaces, imágenes, PR, correcciones.
Aprobación	Rik (o quien delegue)	Visto bueno final; su nombre queda en `aprobadoPor`.
Infraestructura	Automática	GitHub + Vercel: previews, checks y publicación.

8. Puesta en marcha sugerida

1. **Semana 1:**
   construir `scripts/blog-nuevo.mjs` + `BLOG.md` + `materiales/briefs/`, y entregar la guía al equipo SEO (el otro PDF).
2. **Semana 2:**
   piloto con 1 brief real de SEO, de brief a publicado, midiendo fricción del formato.
3. **Mes 2:**
   si el ritmo es 2+ artículos/mes, activar el agente programado en la nube sobre la bandeja de briefs (nivel 2).
